

PySecPatch: Measuring the Synthetic-to-External Generalization Gap in Python Security Repair

Ahmed Bin Khalid (ORCID: 0000-0002-0616-2604)

Independent Researcher

ahmed.khalid2108@gmail.com

Abstract

Language models specialized for secure coding are commonly evaluated on generated examples that resemble their training distribution, while deployment requires transfer to unfamiliar prompts, full files, and repository-level verification. This paper presents PySecPatch, a defensive Python security model and reproducible evaluation suite designed to measure that gap. PySecPatch is a QLoRA adapter for Qwen2.5-Coder-7B-Instruct trained in two consecutive stages on 56,400 generated training records drawn from a 72,000-record corpus. The corpus spans vulnerability detection, CWE classification, clean negatives, snippet repair, and repository-patch selection. Splits are assigned by template family, exact normalized duplicates are removed, and test and holdout records never enter optimization.

On a 3,200-record family-disjoint holdout, PySecPatch increases classification accuracy from 4.81% for the unmodified base model to 90.72%, classification F1 from 9.18% to 93.40%, security-control pass from 1.63% to 88.08%, and clean-negative preservation from 0% to 100%. A paired comparison yields 2,770 PySecPatch-only correct predictions and 21 base-only correct predictions (exact two-sided McNemar $\log_{10}(p) = -787.25$). Transfer is substantially weaker. On the pinned SALLM external benchmark subset, PySecPatch achieves 26.77% functional pass, 31.88% security-test pass, and 9.58% secure-functional pass. On an 800-record repository-format holdout, 38.00% of generated patches apply and 34.38% pass the security control. In a frozen 24-case end-to-end suite, all vulnerable cases are detected and all clean cases are preserved, but none of 12 proposed patches passes every acceptance gate. The findings show that strong controlled specialization does not imply reliable external secure generation or repository repair. PySecPatch releases the adapter, both dataset stages, raw predictions, frozen manifests, confidence intervals, and failure evidence to support reproducible research on security generalization.

1. Introduction

Secure-code models sit between two evaluation traditions. Vulnerability-detection work measures whether a system identifies known weakness classes, while code-generation work measures whether generated programs satisfy functional tests. A practical repair system must do both and must also produce a change that applies to the current source tree, preserves unrelated behavior, introduces no new dangerous construct, and passes regression tests. These requirements create several distinct failure surfaces that aggregate metrics can obscure.

Generated training data offers precise labels, balanced weakness coverage, controllable negatives, and deterministic reconstruction. It also risks teaching superficial regularities. Family-disjoint splits reduce direct template reuse but do not reproduce the ambiguity, incomplete context, dependency behavior, or patch mechanics of real repositories. Consequently, high held-out accuracy can coexist with poor external transfer.

This work studies that divergence through PySecPatch, a 7B defensive Python security adapter and a layered evaluation protocol. The objective is not to claim that a generated corpus solves automated vulnerability repair. Instead, the study asks how far controlled specialization transfers as evaluation moves from synthetic snippets to independent secure-generation prompts and then to verified repository patches.

The work makes four contributions:

- 1 A two-stage, 72,000-record Python security corpus with family-level splitting, clean negatives, 43 CWE categories in the second stage, and explicit training/evaluation boundaries.

- 2 A reproducible QLoRA specialization of Qwen2.5-Coder-7B-Instruct with a strict JSON repair contract and no automatic publication or secret storage.
- 3 A layered evaluation spanning a paired base comparison, internal test and holdout splits, SALLM functional and security checks, repository-format tasks, and a frozen end-to-end acceptance suite.
- 4 An empirical failure analysis demonstrating that large gains in controlled classification and snippet repair do not translate proportionally to external secure-functional generation or applicable repository patches.

2. Background and Related Work

Qwen2.5-Coder is a code-specialized model family evaluated across code generation, completion, reasoning, and repair tasks [1]. Parameter-efficient adaptation provides a practical route to task specialization. LoRA represents parameter updates through low-rank matrices [2], while QLoRA combines low-rank adaptation with quantized base weights to reduce memory requirements [3]. PySecPatch uses 4-bit NF4 double quantization and LoRA rank 16.

SecurityEval introduced a Python dataset covering 75 CWE types for assessing insecure code generation [4]. SALLM extended security-centered code evaluation by combining compilation, functional testing, and security assessment over Python prompts [5]. PySecPatch uses a pinned SALLM revision as its independent secure-generation evaluation and reports missing upstream fixtures rather than silently converting infrastructure limitations into model failures.

RealVuln evaluates scanners on intentionally vulnerable Python repositories with manually labeled vulnerabilities and false-positive traps [6]. The PySecPatch repository includes a rule-based AST and Bandit scanner evaluated against RealVuln. That result is kept separate from adapter evaluation because the current RealVuln runner does not invoke the neural model. This distinction prevents a front-end scanner score from being attributed to model specialization.

LineVul demonstrated transformer-based function classification and line localization on C/C++ vulnerability data [7]. DiverseVul subsequently showed that larger datasets do not eliminate high false-positive rates or unseen-project generalization problems [8]. PrimeVul tightened labeling, deduplication, chronological splitting, and realistic evaluation and found that conventional benchmarks can substantially overestimate code-model vulnerability detection [9]. These results motivate PySecPatch's family-level splits, explicit clean negatives, and separate localization metric, while also limiting what its generated Python distribution can establish.

For repair, Pearce et al. evaluated zero-shot language-model patches across generated, hand-crafted, and historical vulnerabilities and found a marked gap between controlled and real-world functional correctness [10]. VulRepair used T5 for learned vulnerability repair and provided a reproducible task-specific baseline [11]. PySecPatch extends this line of inquiry by reporting contract validity, security-control success, patch application, and complete repository acceptance as distinct outcomes.

Prior vulnerability-repair work has reported exact patch generation and test-based validation, but repository repair imposes constraints absent from snippet tasks: correct file selection, byte-accurate context, syntactically valid unified diffs, dependency preservation, and regression behavior. The present study exposes these stages separately rather than collapsing them into a single repair score. This verification-centered framing is also consistent with the NIST Secure Software Development Framework's emphasis on defined security requirements, review, testing, vulnerability response, and retained release evidence [12].

3. Research Questions

RQ1: How much does two-stage security specialization improve controlled Python vulnerability triage and snippet repair relative to the unmodified base model?

RQ2: How well do controlled improvements transfer to an independent secure-code generation benchmark?

RQ3: Which failures dominate when the same model is required to produce verifiable repository patches?

4. PySecPatch

4.1 Defensive contract

The model receives Python code and returns a strict JSON object with vulnerability status, CWE, vulnerability type, vulnerable lines, explanation, fixed code, patch summary, and a safe test. Repository-patch tasks use a separate three-field contract containing the finding identifier, summary, and unified diff. Invalid JSON is retained as evidence rather than repaired silently.

The repository agent accepts only local repositories or GitHub repositories explicitly supplied by the user. Its static checks cover command execution, dynamic evaluation, unsafe YAML and pickle use, suspicious SQL construction, traversal-style file access, and secret-like assignments. Candidate patches are evaluated in an isolated copy. Acceptance requires a nonempty relevant-file diff, Python parsing, test success when tests exist, no increase in Bandit findings, no introduced banned pattern, and preservation of unrelated files.

4.2 Training corpus

Training proceeded in two stages. Stage A contains 12,000 records across 35 CWEs and 40 generator profiles: 8,400 train and 1,200 each for validation, test, and holdout. Stage B contains 60,000 records across 43 CWEs and 50 profiles: 48,000 train and 4,000 each for validation, test, and holdout. Stage B comprises 24,000 detection records, 24,000 repair records, and 12,000 repository-patch records. Across both stages, 56,400 records enter optimization and 15,600 remain outside training.

All examples are generated Python and released under Apache-2.0. No random GitHub scraping or unknown-license external dataset enters training. Each record carries an identifier, split, family, CWE, task metadata, source, and license. Exact duplicate and normalized-code hashes are checked before writing outputs. Cross-stage auditing finds zero exact identifiers and zero exact normalized code/fix pairs. It identifies 702 shared abstract structures, which are retained and disclosed because structural similarity is not equivalent to exact duplication but may reduce distributional independence.

4.3 Split design

Rows are not independently randomized. Template families are assigned to splits so that test and holdout code shapes differ from training families. Stage B holdout contains 4,000 records: 1,600 detection, 1,600 repair, and 800 repository-patch cases. The analysis evaluation uses the 3,200 detection and repair records; repository-patch records are scored separately.

4.4 Optimization

Stage A uses two epochs, learning rate $2e-4$, effective batch size 8, maximum sequence length 4,096, paged AdamW 8-bit, LoRA rank 16, and alpha 32. Stage B continues from the Stage A adapter for one epoch using learning rate $1e-4$, effective batch size 16, maximum sequence length 2,048, the same optimizer, rank, and alpha, and no sequence packing. Stage B training uses 4-bit NF4 double quantization. The run completes 3,000 optimizer steps with training loss 0.1284 and validation loss 0.3163. Test and holdout records are not loaded by the trainer.

5. Evaluation Method

5.1 Controlled paired comparison

The base model and final adapter are evaluated on identical Stage B holdout records using the same prompt, greedy generation contract, output limit of 900 tokens, batch size 8, and seed. Metrics include accuracy, balanced accuracy, precision, recall, F1, MCC, strict JSON, schema validity, CWE exact match, parseability, structural and normalized exact repair, security-control pass, dangerous-regression avoidance, clean preservation, and line-localization F1.

Differences are computed per record. Confidence intervals use 10,000 paired bootstrap resamples clustered by the 50 holdout template families. Classification discordance is tested with an exact two-sided McNemar test.

5.2 SALLM

The external evaluation uses SALLM revision `0159a63daed0a88f461bbd69dd1160893e394a67`. Generation uses 100 prompts, ten samples per prompt, temperature 0.2, a 700-token output limit, and seed 20260619. Functional tests run in ten isolated GitHub Actions shards. Generated files are scanned with the disclosed CodeQL security-extended configuration. Four prompts lack test fixtures at the pinned upstream revision; results therefore report the 96-prompt scored subset, full-benchmark bounds, and the missing identifiers.

5.3 Repository-format evaluation

The Stage B repository task asks the model to select a finding-specific unified diff while preserving unrelated files. Test and holdout each contain 800 records. Metrics cover finding-ID accuracy, strict JSON, relevant-file restriction, patch application, parseability, exact patched code, security-control pass, and dangerous-regression avoidance.

5.4 Frozen end-to-end suite

The Stage 6 suite was generated after model freeze and contains 12 paired vulnerable/clean repository cases across six CWE categories and seven application styles. It is frozen by SHA-256 before inference. The full agent scans each repository, invokes the model, attempts the patch, and applies the complete acceptance gate. A second configuration permits two verifier-guided retries while preserving the suite, model, and verifier.

6. Results

6.1 RQ1: controlled specialization

Metric	Base	PySecPatch	Difference
Accuracy	4.81%	90.72%	+85.91 pp
Balanced accuracy	3.21%	93.81%	+90.60 pp
Precision	16.14%	100.00%	+83.86 pp
Recall	6.42%	87.63%	+81.21 pp
F1	9.18%	93.40%	+84.22 pp
Strict JSON	91.53%	99.44%	+7.91 pp
Schema validity	5.16%	99.44%	+94.28 pp
Clean preservation	0.00%	100.00%	+100.00 pp
Security-control pass	1.63%	88.08%	+86.46 pp
Normalized exact repair	0.17%	83.33%	+83.17 pp
Parseable fixed code	6.42%	99.21%	+92.79 pp

The final model is correct on 2,770 records missed by the base, while the base is correct on 21 records missed by the final model. The exact two-sided McNemar result is $\log_{10}(p) = -787.25$. Family-clustered 95% bootstrap intervals for the final-minus-base differences are [79.38, 91.78] percentage points for accuracy, [3.16, 13.66] for strict JSON, [90.97, 97.16] for schema validity, [100.00, 100.00] for clean preservation, [79.58, 92.46] for security-control pass, and [74.88, 90.42] for normalized exact repair. These intervals exclude zero for every reported headline difference. The results answer RQ1 affirmatively within the controlled distribution: specialization substantially improves contract adherence, classification, negative restraint, and reference-aligned snippet repair.

Line-localization F1 reaches only 0.4407, substantially below classification and CWE accuracy. The model often recognizes the weakness and produces the expected repair while identifying an incomplete or differently scoped line set.

6.2 RQ2: external transfer

SALLM results are much lower: 26.77% functional pass, 31.88% security-test pass, and 9.58% secure-functional pass on 960 scored samples. Secure pass@1 is 9.58%, pass@3 is 11.51%, and pass@5 is 12.26%. CodeQL reports 62.60% of generated files as finding-free under the disclosed query suite.

The external result does not track the controlled holdout improvement. In particular, several weakness groups show high functional or security success but not both. This confirms that secure generation is conjunctive: a sample that is safe but functionally wrong, or functional but still vulnerable, fails the secure-functional objective.

6.3 RQ3: repository repair

Repository-format test and holdout patch-application rates are 35.25% and 38.00%. Security-control pass rates are 31.88% and 34.38%, respectively. Exact patched-code rates remain near 30%. Finding identifiers, JSON structure, and relevant-file restrictions exceed 99%, showing that the model follows the high-level contract while often failing the mechanics or semantics of the patch.

In Stage 6, all 12 vulnerable cases are detected, all 12 clean cases are preserved, and no unrelated file is rewritten. Nevertheless, zero proposed patches pass the complete acceptance gate. Failures include context that does not match the current file, malformed hunks, an empty patch, trailing whitespace rejection, and a patch that fails tests. Two verifier-guided retries increase runtime from 70 to 186 seconds but do not improve acceptance. Retry prompting is therefore insufficient when the underlying representation remains unconstrained free-form diff generation.

7. Discussion

7.1 Controlled success is real but narrow

The paired base comparison shows that the adapter learned the output contract and the controlled security transformations. Family-disjoint splitting and the large base gap make simple row memorization an incomplete explanation. At the same time, the corpus is generated from finite profile families. The results establish specialization within that designed space, not general Python security competence.

7.2 Detection, triage, and repair should be separated

PySecPatch is most defensible as a conservative triage and candidate-repair assistant. On controlled holdout data it produces no false positives and preserves all clean negatives, but it misses 297 of 2,400 vulnerable records. The repository front end remains rule-based; its RealVuln result must not be attributed to the adapter. A future neural RealVuln evaluation requires a separately validated file-chunking and localization protocol.

7.3 Free-form unified diffs are a bottleneck

Stage 6 failures suggest a systems intervention rather than another retry prompt. Structured edits, exact source-span replacement, deterministic diff construction, and verifier-guided search can separate semantic repair generation from patch serialization. Such changes can improve the agent without altering model weights, but they constitute a new system configuration and must be evaluated separately.

7.4 Implications for benchmark reporting

Reporting only the controlled holdout would imply a mature repair capability. Reporting only Stage 6 would hide substantial gains in classification and snippet repair. Layered evaluation reveals both. Security-model releases should publish independent functional-security results, clean-negative behavior, raw outputs, repository acceptance, and infrastructure exclusions together.

8. Threats to Validity

Construct validity. Exact repair metrics reward agreement with a reference patch even when another repair may be safe. Security-control tests cover intended controls but cannot prove absence of all vulnerabilities. CodeQL results depend on the disclosed query revision.

Internal validity. Both data stages are produced by deterministic generators. Although splits are family-disjoint and exact duplicates are removed, 702 shared abstract structures remain across stages. Batch execution occurred on different but comparable NVIDIA GPUs; model, prompt, seed, and batch size were held fixed for the paired holdout.

External validity. Generated snippets do not reproduce complete dependency graphs, repository conventions, or maintainer intent. SALLM supplies independent prompts but remains a bounded benchmark. Stage 6 contains only 24 cases and six CWEs, producing wide confidence intervals.

Conclusion validity. Paired bootstrap resampling clusters by template family and McNemar testing uses paired predictions. Multiple exploratory per-CWE observations are descriptive and should not be interpreted as individually corrected hypothesis tests.

9. Ethics and Responsible Release

PySecPatch is restricted to defensive analysis of local repositories or GitHub repositories explicitly supplied by a user. The project does not automate unauthorized scanning, exploit chains, credential theft, malware, persistence, phishing, or evasion. Generated patches require human review. The release includes limitations and failed cases because overstating repair reliability can create security risk.

Training data are generated and Apache-2.0 licensed. No secrets are stored; optional service credentials are read only from environment variables. The base model license was audited before training, and base weights are not redistributed.

10. Reproducibility and Availability

The release includes source code, model adapter, both dataset stages, split statistics, contamination reports, raw prediction files, paired-comparison code, frozen suite hashes, SALLM shard evidence, package versions, and SHA-256 manifests. GitHub hosts the code and concise reports; Hugging Face hosts the adapter and dataset configurations. Zenodo concept DOI 10.5281/zenodo.21015503 resolves to the latest archived software release [13]. The independently archived PySecPatch-72K dataset is identified by DOI 10.5281/zenodo.21016753 [14]. The manuscript, citation metadata, and machine-readable CodeMeta record are distributed with the release.

11. Conclusion

PySecPatch demonstrates that focused QLoRA training can transform a general code model into a highly accurate controlled Python security classifier and snippet-repair assistant. The same adapter remains unreliable for external secure-functional generation and autonomous repository patching. The gap is not a peripheral limitation; it is the main empirical result. By releasing the successful and failed evaluations together, PySecPatch provides a reproducible basis for studying how data design, structured editing, and verification can close the distance between benchmark specialization and dependable security repair.

References

- [1] B. Hui et al. "Qwen2.5-Coder Technical Report." arXiv:2409.12186, 2024.
- [2] E. Hu et al. "LoRA: Low-Rank Adaptation of Large Language Models." ICLR, 2022.
- [3] T. Dettmers et al. "QLoRA: Efficient Finetuning of Quantized LLMs." NeurIPS, 2023.
- [4] M. L. Siddiq and J. C. S. Santos. "SecurityEval Dataset: Mining Vulnerability Examples to Evaluate Machine Learning-Based Code Generation Techniques." MSR4P&S, 2022. DOI: 10.1145/3549035.3561184.

- [5] M. L. Siddiq, J. C. S. Santos, S. Devareddy, and A. Muller. "SALLM: Security Assessment of Generated Code." ASE Workshops, 2024. DOI: 10.1145/3691621.3694934.
- [6] J. Pellew and F. Raza. "RealVuln: Benchmarking Rule-Based, General-Purpose LLM, and Security-Specialized Scanners on Real-World Code." arXiv:2604.13764, 2026.
- [7] M. Fu and C. Tantithamthavorn. "LineVul: A Transformer-based Line-Level Vulnerability Prediction." MSR, 2022. DOI: 10.1145/3524842.3528452.
- [8] Y. Chen, Z. Ding, L. Alowain, X. Chen, and D. Wagner. "DiverseVul: A New Vulnerable Source Code Dataset for Deep Learning Based Vulnerability Detection." RAID, 2023. DOI: 10.1145/3607199.3607242.
- [9] Y. Ding, Y. Fu, O. Ibrahim, C. Sitawarin, X. Chen, B. Alomair, D. Wagner, B. Ray, and Y. Chen. "Vulnerability Detection with Code Language Models: How Far Are We?" arXiv:2403.18624, 2024.
- [10] H. Pearce, B. Tan, B. Ahmad, R. Karri, and B. Dolan-Gavitt. "Examining Zero-Shot Vulnerability Repair with Large Language Models." IEEE Symposium on Security and Privacy, 2023.
- [11] M. Fu, C. Tantithamthavorn, T. Le, V. Nguyen, and D. Phung. "VulRepair: A T5-Based Automated Software Vulnerability Repair." ESEC/FSE, 2022. DOI: 10.1145/3540250.3549098.
- [12] M. Souppaya, K. Scarfone, and D. Dodson. "Secure Software Development Framework (SSDF) Version 1.1." NIST SP 800-218, 2022. DOI: 10.6028/NIST.SP.800-218.
- [13] A. Bin Khalid. "PySecPatch: Defensive Python Vulnerability Triage and Repair Research Artifacts." Zenodo, 2026. DOI: 10.5281/zenodo.21015503.
- [14] A. Bin Khalid. "PySecPatch-72K: A Family-Disjoint Python Security Triage and Repair Corpus." Zenodo, 2026. DOI: 10.5281/zenodo.21016753.